Stripe payments

Verifying what was taken, and reading the log when something fails

For the AngelFoodsAPI behind angelfood.co.nz. Written 11 October 2026, against the live API.

Where things stand today

Nothing is being logged. Webhooks are rejected before they are read.

A live check of `POST /api/Stripe/Webhook` returns **HTTP 503** with the message *"Stripe:WebhookSecret is not configured."* Until that secret is set, every event Stripe sends is refused and never reaches the log table, so the queries later in this document will return nothing. Fixing it is the first section below.

What that means in practice

What should happen when a card clears	What happens now
The order is marked **Paid**	It stays Pending
The customer gets a confirmation email	No email is sent
An internal notification goes out	No notification
GoSweetSpot books the courier (once enabled)	Never runs
The event is written to StripePaymentLogs	Nothing is written

A second thing worth knowing: `GET /api/Stripe/Config` reports `"liveMode": false`. The API is on **test keys**, so no real money can move yet. That is the right place to be while proving this out — but it does mean the takings figures will read zero until live keys are in place.

Switching the logging on

Three steps, all in the Stripe Dashboard and your server configuration. No code change is needed — the endpoint is already written, open and anonymous, so no firewall change is needed either.

1. Add the endpoint in Stripe

Dashboard → **Developers** → **Webhooks** → **Add endpoint**, pointing at:

```
https://angelfood-api.webappconsulting.com.au/api/Stripe/Webhook
```

2. Subscribe it to these events

These are the ones the handler acts on. Anything else is logged but ignored.

Event	What the API does with it
`checkout.session.completed`	Marks the order paid; sends the emails
`payment_intent.succeeded`	Same payment, second notice — logged, acted on once
`checkout.session.async_payment_succeeded`	A delayed method finally cleared
`payment_intent.payment_failed`	Logged as a failure; order left Pending
`checkout.session.async_payment_failed`	Same, for delayed methods
`charge.refunded`	Marks the order refunded

Stripe sends both `checkout.session.completed` and `payment_intent.succeeded` for a single payment. Both are logged; only the first one changes anything, so an order cannot be marked paid twice or email the customer twice.

3. Set the signing secret on the server

Copy the signing secret from that endpoint's page in Stripe — it begins `whsec_` — and set it as an **environment variable on the server**:

```
Stripe__WebhookSecret = whsec_xxxxxxxxxxxxxxxxxxxx
```

> **Not in appsettings.json.**
>
> That file is committed to source control, so a secret placed there is shared with everyone who can read the repository and stays in its history after removal. The double underscore is how .NET reads a nested setting from the environment. The same applies to the Stripe secret key.

The signature is the only thing that makes an incoming payment notification believable: the webhook endpoint is open to the internet, so without it anyone could post a fake payment and mark an order paid. That is why the handler refuses to run rather than trusting what arrives.

Verifying payments

Three endpoints already exist for this. All require a bearer token and return **401** without one, so sign in first via `POST /api/Auth` and use the token it returns.

Every event, newest first

```
curl -H "Authorization: Bearer TOKEN" \
"https://angelfood-api.webappconsulting.com.au/api/Stripe/Payments?pageSize=50"
```

Filters, all optional and combinable:

Parameter	Use
`orderNumber`	Everything that happened to one order
`eventType`	One kind of event, e.g. payment_intent.payment_failed
`liveMode`	true for real money, false for test. Worth always setting
`from / to`	A date range, on when we recorded it
`page / pageSize`	Paging. pageSize is capped at 200, default 50

Test and live events share one table, so a total that mixes them is misleading. Set `liveMode` deliberately whenever the number matters.

Failures only

```
curl -H "Authorization: Bearer TOKEN" \
"https://angelfood-api.webappconsulting.com.au/api/Stripe/Payments\
?eventType=payment_intent.payment_failed&liveMode=true"
```

A running total

```
curl -H "Authorization: Bearer TOKEN" \
"https://angelfood-api.webappconsulting.com.au/api/Stripe/Payments/Summary"
```

Field	Meaning
`payments`	Completed checkouts with status paid
`takings`	Sum of those amounts
`refunds / refunded`	Count and value of refunds
`failures`	Payment attempts Stripe rejected
`unhandled`	Events that arrived but our side failed to process. Watch this one

The summary counts **live mode only**, deliberately, so the figure means the same thing as the bank statement. It takes optional `from` and `to` dates.

Reading one event in full

The list leaves out `RawPayload` on purpose — a hundred events each carrying Stripe's complete JSON helps nobody. To get one event entire, including that payload, fetch it by its id:

```
curl -H "Authorization: Bearer TOKEN" \
"https://angelfood-api.webappconsulting.com.au/api/Stripe/Payments/123"
```

What every row records

Field	What it tells you
`EventId`	Stripe's own id for the event. Use it to find the event in their dashboard
`EventType`	Which of the events above this was
`LiveMode`	Real money, or a test
`OrderNumber`	Our order, when the event names one
`WebsiteSalesOrderId`	The order row, for joining
`SessionId`	The Stripe Checkout session
`PaymentIntentId`	The payment attempt. Also written onto the order as PaymentReference
`ChargeId`	The charge, once money moved
`Amount`	What was taken, in dollars
`AmountRefunded`	What has since gone back

Field	What it tells you
`Currency`	nzd
`Status`	Stripe's status for the payment, e.g. paid
`CardBrand, CardLast4`	Enough to identify the card to a customer, and no more
`CustomerEmail`	As given at checkout
`FailureCode`	Stripe's machine-readable reason, when it declined
`FailureMessage`	The same reason in words
`Handled`	Whether our side processed it successfully
`HandlingError`	If not, why
`EventDate`	When Stripe raised it
`CreatedDate`	When we recorded it
`RawPayload`	Stripe's original JSON, on the single-event endpoint only

When something fails

There are two different kinds of failure, and they are read in different places.

The customer's card was declined

The event is `payment_intent.payment_failed`. `FailureCode` and `FailureMessage` carry Stripe's reason. The order is deliberately left **Pending** rather than marked failed, because the customer can try another card against the same order, and the failed attempt is already recorded as its own row.

The payment worked, but our side did not

This is the one to watch. The row will have `Handled: false` and a `HandlingError` explaining what went wrong — a mail server refusing connections, for instance.

> **Stripe reporting success does not mean it worked.**
>
> The webhook answers **HTTP 200 even when our handling fails**, because a non-200 makes Stripe retry for days over something a retry cannot fix. So the Stripe dashboard will show the event as delivered either way. `Handled` is the field that tells you the truth, and `unhandled` in the summary is the number worth alerting on.

Everything that happened to one order

```
curl -H "Authorization: Bearer TOKEN" \
"https://angelfood-api.webappconsulting.com.au/api/Stripe/Payments?orderNumber=AF-10432"
```

This is usually the fastest way to answer "what happened to this customer's payment?" — it returns the whole sequence, attempt by attempt, in order.

Testing it before going live

Once the webhook secret is set, place test orders while the API is still on test keys. Stripe publishes cards that produce specific outcomes:

Card number	What it does	What to check
`4242 4242 4242 4242`	Succeeds	Order goes Paid, confirmation email sent, row logged with Handled true
`4000 0000 0000 0002`	Declined outright	Row logged with a FailureCode, order still Pending
`4000 0000 0000 0341`	Attaches, then fails on charge	The useful one: proves failures after checkout are captured
`4000 0025 0000 3155`	Requires 3D Secure	Confirms the authentication flow completes

Any future expiry date and any CVC will do. Stripe's dashboard has a **Send test webhook** button on the endpoint page, which is the quickest way to confirm the secret is right before involving a card at all.

If it still does not work

What you see	What it usually means
503, "WebhookSecret is not configured"	The environment variable is not set, or the app was not restarted after setting it
400, "Signature verification failed"	The secret does not match this endpoint. Each endpoint in Stripe has its own — check you copied the right one
Stripe shows delivered, nothing in the log	Almost always the two above. The log is only written after the signature verifies
Rows logged, but Handled is false	The event arrived and verified; our processing failed. Read HandlingError
Order paid in Stripe but Pending here	Either no webhook arrived, or it arrived and failed. Search the log by orderNumber
Takings read zero	The summary counts live mode only, and the API is currently on test keys
401 from the Payments endpoints	No bearer token. Sign in at POST /api/Auth first

One structural note

The API runs database migrations on startup. A migration that fails takes the whole API down rather than breaking one feature — which is what the 500.30 outage on 11 October was. It is worth considering applying migrations as a deployment step instead, so that a bad migration fails the deployment rather than the service.